

澄清眼科 AI 護眼精靈 — 系統交接文件

專案倉庫：[TlimLin/ChengChing-AI-EyeCare](#) 文件撰寫日期：2026-05-20 版本：基於 main branch
commit [69dbc97](#)

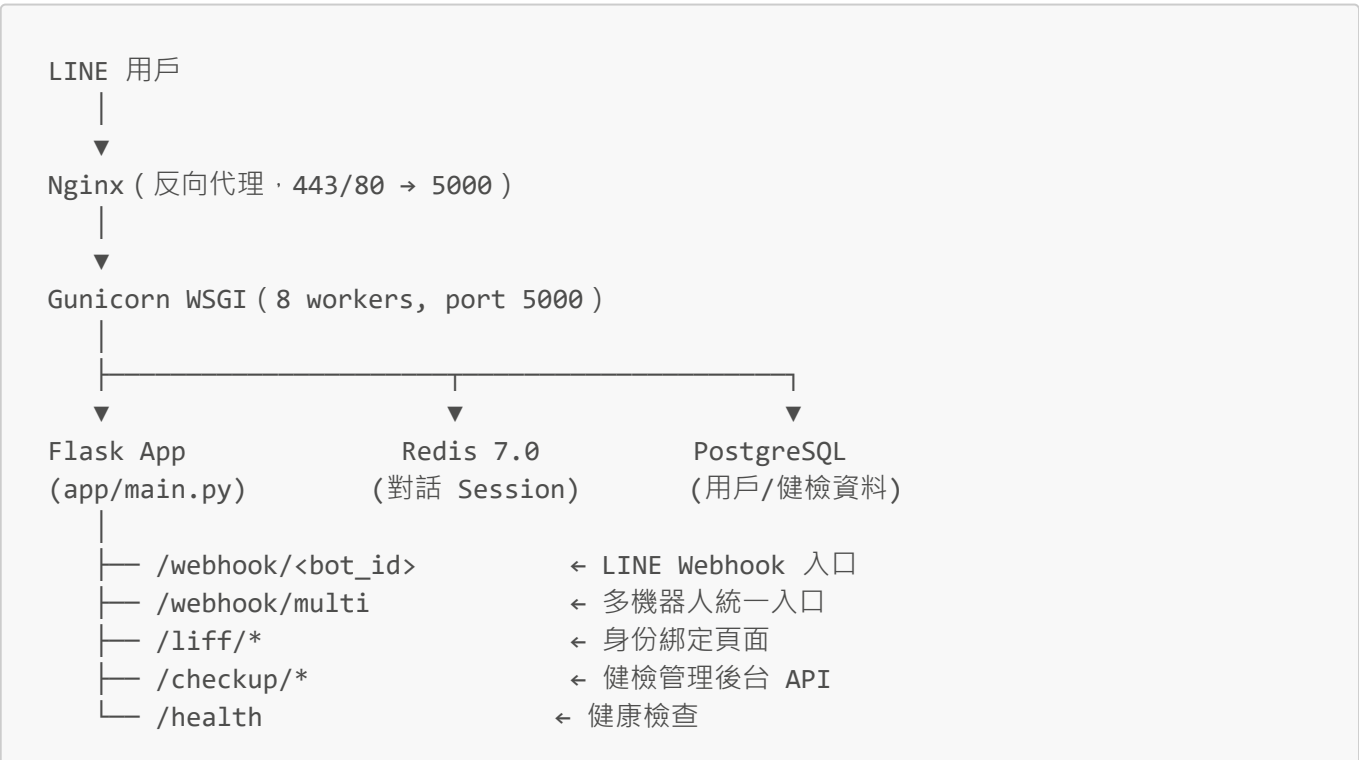
一、專案概述

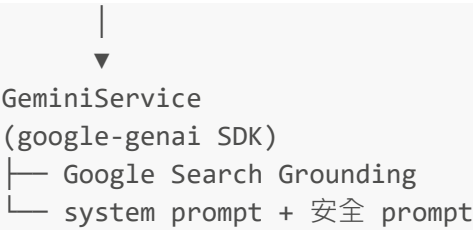
澄清眼科 AI 護眼精靈是一套以 LINE Official Account 為前端、Flask + Gunicorn 為後端的智慧客服系統，核心 AI 能力由 **Google Gemini 2.5 Flash** 提供，搭配 **Google Search Grounding** 即時查詢澄清眼科官網（[cceye.com.tw](#)）資料，並透過 **Redis** 跨 Worker 共享對話歷史。

主要功能模組

功能模組	說明
通用眼科 AI 問答	Gemini + Google Search，保留 5 輪上下文
智慧分流 (Triage)	AI 判斷是否為症狀，推薦對應醫師
健檢報告摘要	串接 Google Sheets 健檢資料，Gemini 生成摘要
社區健檢管理後台	健檢資料匯入、異常通知、查詢介面
就診提醒排程	APScheduler 每日 09:30 推播提醒
身份綁定 (LIFF)	LINE Front-end Framework 使用者實名綁定
Prompt 安全防護	三層防護：輸入攔截 → 特徵比對 → AI 審查

二、系統架構





三、專案目錄結構

```
ChengChing-AI-EyeCare/
├── app/
│   ├── main.py                # Flask 工廠函數：create_app()
│   ├── config/
│   │   ├── setting.py        # Flask Config ( 讀取 .env )
│   │   └── bot_config.py     # BotConfigManager：讀取 bot_configs.json +
環境變數
│   └── prompts/
│       ├── cceye_system_prompt.txt # 澄清眼科主要 System Prompt
│       ├── triage_system_prompt.txt # 智慧分流 System Prompt
│       └── security_prompt.txt     # 安全防護 Prompt
│   ├── routes/
│       ├── multi_webhook.py      # LINE Webhook 路由
│       ├── liiff_routes.py      # LIFF 身份綁定路由
│       └── checkup_routes.py     # 健檢管理後台 API ( 最大模組 · 27KB )
│   ├── services/
│       ├── gemini_service.py     # Gemini AI 通用服務 ( 含安全防護 )
│       ├── multi_linebot.py     # LINE Bot 核心邏輯 + 訊息處理
│       ├── triage_service.py    # 智慧分流服務 ( 繼承 GeminiService )
│       ├── doctor_service.py    # 醫師資料查詢與推薦
│       ├── token_tracker.py     # Token 用量計算與 Grounding 配額追蹤
│       ├── health_check_service.py # Google Sheets 健檢資料查詢
│       ├── health_summary_service.py # AI 生成健檢摘要
│       ├── appointment_reminder.py # 就診提醒排程邏輯
│       ├── checkup_import_service.py # 健檢資料匯入 ( Excel → DB )
│       ├── checkup_notify_service.py # 異常健檢通知推播
│       ├── abnormal_import_service.py # 異常資料匯入處理
│       ├── data_cleaner.py      # 資料清洗工具
│       ├── his_service.py      # HIS 系統資料橋接
│       └── workbook_reader.py  # Excel 活頁簿讀取工具
│   ├── models/
│       ├── database.py         # PostgreSQL 連線池 ( psycopg2 )
│       ├── user.py            # User 資料模型
│       └── user_storage.py     # 使用者 CRUD ( 含綁定狀態 )
│   └── utils/
│       └── paths.py            # 全域路徑常數 ( LOGS_DIR, APP_LOG... )
├── config/
│   ├── gunicorn.conf.py       # Gunicorn 生產配置 ( 8 workers, timeout=120 )
│   └── systemd/
│       └── cceye-linebot.service.example
```

```

├── frontend/
│   └── liff/
│       └── bind/
│           └── index.html          # LIFF 身份綁定前端頁面
├── tools/
│   ├── create_admin.py           # 建立後台管理員帳號
│   ├── find_place_ids.py         # 查詢 Google Maps data_id
│   └── google_review_response_time.py # Google 評論回覆時間分析
├── specs/                        # 功能規格文件
│   ├── 001-web-eye-exercise/
│   ├── 002-line-bot-rag/
│   ├── 003-line-bot-triage/
│   ├── 004-line-bot-reminders/
│   └── 005-line-bot-appointment/
├── requirements.txt              # Python 依賴
├── run.py                        # 開發模式入口
├── .gitignore
└── README.md

```

3.1 功能模組對應程式架構

以下依功能分類，列出各功能實際涉及的程式檔案：

● 一般衛教諮詢 (通用 AI 問答)

```

app/services/gemini_service.py      # Gemini AI 通用服務 ( 含安全防護 )
app/services/multi_linebot.py       # LINE Bot 核心邏輯 + 訊息處理
app/services/token_tracker.py       # Token 用量計算與 Grounding 配額追蹤
app/config/prompts/cceye_system_prompt.txt # 澄清眼科主要 System Prompt
app/routes/multi_webhook.py         # LINE Webhook 路由入口

```

● AI 症狀分流 (智慧分流 + 醫師推薦)

```

app/services/gemini_service.py      # Gemini AI 通用服務 ( 含安全防護 )
app/services/multi_linebot.py       # LINE Bot 核心邏輯 + 訊息處理
app/services/triage_service.py      # 智慧分流服務 ( 繼承 GeminiService )
app/services/doctor_service.py      # 醫師資料查詢與推薦
app/routes/multi_webhook.py         # LINE Webhook 路由入口

```

● 健檢報告摘要 (暫定不會使用，可忽略)

```

app/services/health_check_service.py    # Google Sheets 健檢資料查詢
app/services/health_summary_service.py  # AI 生成健檢摘要
app/services/multi_linebot.py           # LINE Bot 核心邏輯 ( 健檢關鍵字觸發 )
app/routes/multi_webhook.py             # LINE Webhook 路由入口

```

● 社區健檢管理後台

```

app/routes/checkup_routes.py            # 健檢管理後台 API ( 最大模組 · 27KB )
app/services/checkup_import_service.py  # 健檢資料匯入 ( Excel → DB )
app/services/checkup_notify_service.py  # 異常健檢通知推播
app/services/abnormal_import_service.py # 異常資料匯入處理
app/services/data_cleaner.py            # 資料清洗工具
app/services/workbook_reader.py         # Excel 活頁簿讀取工具
app/models/database.py                  # PostgreSQL 連線池

```

● 就診提醒排程

```

app/services/appointment_reminder.py    # 就診提醒排程邏輯 ( APScheduler )
app/services/his_service.py              # HIS 系統資料橋接
config/gunicorn.conf.py                  # post_fork() 確保僅單一 Worker 啟動排程

```

● 身份綁定 (LIFF)

```

app/routes/liff_routes.py                # LIFF 身份綁定路由
app/models/user.py                       # User 資料模型
app/models/user_storage.py               # 使用者 CRUD ( 含綁定狀態 )
app/models/database.py                   # PostgreSQL 連線池
frontend/liff/bind/                      # LIFF 前端靜態頁面

```

○ 共用基礎設施

```

app/main.py                             # Flask 工廠函數 : create_app()
app/config/setting.py                    # Flask Config ( 讀取 .env )
app/config/bot_config.py                  # BotConfigManager : bot 設定管理
app/utils/paths.py                       # 全域路徑常數
app/models/database.py                   # PostgreSQL 連線池 ( psycopg2 )
config/gunicorn.conf.py                  # Gunicorn 生產配置

```

四、環境需求與安裝

4.1 環境連線資訊

虛擬機

項目	值
帳號	ubuntu
密碼	Cceye@dmin07
IP	192.168.10.214
部署路徑	/home/ubuntu/ChengChing-AI-EyeCare/

```
ssh ubuntu@192.168.10.214
# 密碼：Cceye@dmin07
```

資料庫

項目	值
服務類型	PostgreSQL
主機名	localhost
端口	5432
使用者名稱	eyecare_user
密碼	Cceye@dmin07
資料庫	eyecare

```
psql -h localhost -U eyecare_user -d eyecare
# 密碼：Cceye@dmin07
```

4.2 系統需求

項目	版本
作業系統	Ubuntu 22.04 LTS
Python	3.12.3+
Redis	7.0.15+
PostgreSQL	任意現代版本 (建議 14+)
RAM	建議 4GB+ (生產環境 8 workers)

4.3 Python 套件 (requirements.txt)

```
flask==3.0.0
line-bot-sdk==3.17.1
python-dotenv==1.0.0
google-genai          # Gemini SDK ( 無版本固定 )
gunicorn
redis==5.0.1
gsread==6.0.0         # Google Sheets 存取
google-auth==2.27.0
APScheduler>=3.10.0   # 就診提醒排程
requests>=2.32.0
psycopg2-binary>=2.9.9 # PostgreSQL 驅動
xlrd>=2.0.1
openpyxl>=3.1.0
```

4.4 安裝步驟

```
git clone https://github.com/TIimLin/ChengChing-AI-EyeCare.git
cd ChengChing-AI-EyeCare

# 建立虛擬環境 ( 推薦 uv )
uv venv && source .venv/bin/activate
uv pip install -r requirements.txt

# 安裝並啟動 Redis
sudo apt install redis-server -y
sudo systemctl enable --now redis-server

# 複製並填寫環境變數
cp .env.example .env
vim .env

# 初始化 DB + 測試啟動
python3 run.py
# 訪問 http://localhost:5000/health → {"status":"healthy"}
```

五、環境變數說明

所有機密資料集中於 `.env` (絕對不可 commit) :

```
# — LINE Bot —
CCEYE_LINE_CHANNEL_ACCESS_TOKEN=...
CCEYE_LINE_CHANNEL_SECRET=...

# — Gemini —
CCEYE_GEMINI_API_KEY=...

# — Redis —
```

```

REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0
REDIS_PASSWORD=          # 若無密碼留空

# — PostgreSQL —
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=cceye
POSTGRES_USER=...
POSTGRES_PASSWORD=...

# — Flask —
BASE_URL=https://your-domain.com
DEBUG=false
HOST=0.0.0.0
PORT=5000
FLASK_SECRET_KEY=...

# — 其他整合 —
GOOGLE_SHEET_ID=...          # 健檢報告 Google Sheets ID
GOOGLE_CREDENTIALS_PATH=config/google_credentials.json
LIFF_ID=...                  # LINE LIFF App ID
BOT_CONFIG_FILE=config/bot_configs.json

```

機器人設定檔：`config/bot_configs.json` 存放非機密欄位 (`bot_id`、`env var` 名稱對應)，機密值從環境變數取得，由 `BotConfigManager` 在啟動時合併。

六、核心模組說明

6.1 Flask 應用 (`app/main.py`)

`create_app()` 工廠函數負責：

1. 讀取 `Config (.env)`
2. 設定日誌 (`RotatingFileHandler 10MB × 5 + stdout`)
3. `init_db()`：PostgreSQL 建表及冪等 migration
4. 註冊三個 Blueprint：`webhook_bp`、`liff_bp`、`checkup_bp`
5. 暴露 `/health`、`/` 端點

6.2 Webhook 路由 (`app/routes/multi_webhook.py`)

路由	說明
<code>POST /webhook/multi</code>	統一入口，以 HMAC-SHA256 Signature 自動識別 bot
<code>POST /webhook/<bot_id></code>	直接指定 <code>bot_id</code> 的入口
<code>POST /webhook/<bot_id>/callback</code>	相容 LINE 平台 callback 路徑

6.3 LINE Bot 核心 (`app/services/multi_linebot.py`)

MultiLineBotService 在初始化時：

- 讀取所有 bot_id 設定，建立 WebhookHandler 及 Configuration 實例
- 初始化 GeminiService (通用對話) 與 TriageService (智慧分流)

訊息處理流程 (使用者傳文字)：



6.4 Gemini 服務 (app/services/gemini_service.py)

GeminiService 主要流程 (get_response())：

1. 第 1 層安全：_pre_filter_input() → 攔截 system injection / jailbreak
2. 載入 Redis session (對話歷史)
3. _should_use_search() → 簡單寒暄或非眼科問題不啟動搜尋
4. 呼叫 Gemini 2.5 Flash API
 - 搜尋模式：GoogleSearch Tool + search_site 關鍵字
 - 非搜尋模式：純語言模型，回應更快
5. 第 2 層安全：_sanitize_output() → Prompt fingerprint 比對
6. 第 3 層安全：_is_role_violation() → 可疑輸入才觸發 AI 審查 (PASS/FAIL)
7. 更新對話歷史 (保留最近 5 輪 = 10 條) → 存回 Redis (TTL 1 小時)
8. _extract_search_sources() → 整理參考連結 (優先官網 > Facebook，過濾其他)
9. 記錄 Token 用量與費用到 log

6.5 資料庫設計 (app/models/database.py)

使用 PostgreSQL + psycopg2 ThreadedConnectionPool (minconn=2, maxconn=10)。

資料表	說明
users	LINE 用戶綁定資料 (line_user_id, name, phone, id_number, is_bound)
community_checkups	社區健檢記錄 (UNIQUE: id_number + exam_date + equipment)
checkup_contacts	健檢聯絡資訊 (FK → community_checkups)
admin_users	後台管理員帳號

6.6 智慧分流 (app/services/triage_service.py)

`TriageService` 繼承 `GeminiService`，專門處理有眼部症狀描述的訊息。流程為：

- 1. `DoctorService.build_prompt_fragment()` 依問題動態生成醫師推薦資料
- 2. 傳入 `extra_context` 讓 Gemini 結合醫師資料給出分科建議
- 3. 同樣啟用 Google Search (搜尋 `cceye.com.tw`) 附上官網資訊

七、生產環境部署

7.1 Gunicorn 設定 (`config/gunicorn.conf.py`)

參數	值	說明
<code>workers</code>	8	對應 t3a.xlarge (4 vCPU × 2)
<code>worker_class</code>	sync	同步 Worker
<code>timeout</code>	120 秒	Gemini 回應最長約 8-15 秒
<code>max_requests</code>	1000	自動重啟防記憶體洩漏
<code>preload_app</code>	True	預載 App (減少 Worker 初始化時間)

排程器注意：`post_fork()` 確保只有 `worker.age == 1` 的第一個 Worker 啟動 `APScheduler` (就診提醒每日 09:30)，避免多 Worker 重複執行。

7.2 Systemd 服務

範例檔位於 `config/systemd/cceye-linebot.service.example`。

```
sudo cp config/systemd/cceye-linebot.service.example \
    /etc/systemd/system/cceye-linebot.service
# 按實際路徑修改 WorkingDirectory / User 後：
sudo systemctl daemon-reload
sudo systemctl enable --now cceye-linebot
```

7.3 部署腳本

`README.md` 中提供 `deploy.sh` 自動化腳本，執行順序：

- 1. `git pull origin main`
- 2. `uv pip install -r requirements.txt`
- 3. `systemctl restart cceye-linebot`
- 4. `curl /health` 健康驗證

八、日誌與監控

日誌檔案	內容
<code>logs/app.log</code>	應用程式主日誌 (含 Token 用量、性能統計)

日誌檔案	內容
logs/access.log	HTTP 訪問記錄 (Gunicorn access_log)
logs/error.log	Gunicorn 錯誤日誌
logs/grounding_usage.log	Google Search 每日使用量 (上限 1500/天)

```
# 即時監看主日誌
tail -f logs/app.log

# 每日 Grounding 用量
cat logs/grounding_usage.log

# Systemd 服務狀態
sudo journalctl -u cceye-linebot.service -f

# Redis Session 狀況
redis-cli KEYS "gemini_session:*"
redis-cli DBSIZE
```

九、安全防護架構 (三層防護)



十、第三方服務整合

服務	用途	設定位置
----	----	------

服務	用途	設定位置
LINE Messaging API	Bot 收發訊息	<code>.env</code> CCEYE_LINE_*
LINE LIFF	身份綁定前端頁面	<code>.env</code> LIFF_ID
Google Gemini 2.5 Flash	AI 對話核心	<code>.env</code> CCEYE_GEMINI_API_KEY
Google Search Grounding	即時搜尋官網	Gemini Tool (code 內)
Google Sheets API	健檢資料讀取	<code>.env</code> GOOGLE_SHEET_ID + 金鑰 JSON
Redis 7.0	對話 Session 儲存	<code>.env</code> REDIS_*
PostgreSQL	使用者/健檢資料	<code>.env</code> POSTGRES_*

十一、常見問題排查

服務啟動失敗

```
sudo systemctl status cceye-linebot.service
sudo journalctl -u cceye-linebot.service -n 50

# 常見原因
# 1. Redis 未啟動 : redis-cli ping → 應返回 PONG
# 2. .env 遺漏必要參數 ( PostgreSQL / Gemini key )
# 3. 虛擬環境路徑不正確
```

LINE Bot 無回應

- 1. 確認 Webhook URL 設定為 `https://your-domain.com/webhook/cceye`
- 2. 驗證 SSL 憑證有效 (LINE 強制 HTTPS)
- 3. 防火牆開放 443/80 port
- 4. 測試 : `curl http://localhost:5000/health`

Redis Session 遺失

- 預設 TTL 1 小時，可在 `gemini_service.py` 的 `_save_session_to_redis()` 調整
- 清除所有 session : `redis-cli EVAL "return redis.call('del', unpack(redis.call('keys', 'gemini_session:*')))" 0`

Grounding 配額耗盡

- 每日免費 1500 次，超過需付費
- 查看今日用量 : `cat logs/grounding_usage.log`
- 調整 `_should_use_search()` 邏輯縮減觸發條件

Token 用量異常

- 對話歷史預設保留 5 輪 (10 條)，可在 `gemini_service.py` 第 `if len(contents) > 10` 處調整

- API 呼叫時實際只帶入最近 **1** 輪歷史 (`contents[-2:]`) 以控制費用
-

十二、開發規範

- 分支策略：`main` 為生產分支；功能開發使用 `feature/xxx`
 - 代碼風格：PEP 8，使用 Type Hints 與 Docstring
 - 環境變數：機密值只放 `.env`，公開設定放 `bot_configs.json`
 - 資料庫遷移：`init_db()` 使用 `ADD COLUMN IF NOT EXISTS` 寫法，直接部署即可
 - 更新服務：`git pull` → `uv pip install -r requirements.txt` → `systemctl restart cc-eye-linebot`
-

本文件依據 [ChengChing-AI-EyeCare](#) main branch (commit 69dbc97) 自動生成，如程式碼有更新請同步修訂。